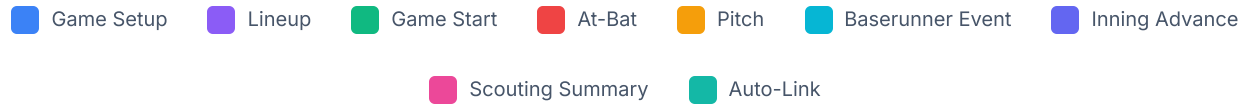


PitchChart — Scouting Game Flow

Full lifecycle: game creation → opponent lineup → live charting → end of game → scouting summary



1

Game Creation

User creates a new scouting game via web or mobile

POST /bt-api/games
`gameService.createGame()`

DATABASE

games **INSERT**

```
charting_mode='scouting',
opponent_name (away team),
scouting_home_team (home team
name), home_team_id (linked user
team), opponent_team_id (FK →
opponent_teams, optional),
game_date, location,
status='scheduled', total_innings,
is_home_game, lineup_size,
created_by (FK → users)
```

opponent_teams **UPDATE**

```
games_played++,
last_game_date=NOW() (if
opponent_team_id provided)
```

Scouting-specific: Both teams are external opponents — `opponent_name` = away team, `scouting_home_team` = home team. The user's own team (`home_team_id`) is used only as the owning team for data organization, *not displayed as a participant*.

Key Inputs

- `charting_mode`: "scouting"
- `opponent_name`: away team free text
- `scouting_home_team`: home team free text
- `opponent_team_id`: UUID from opponent registry (auto-set if user picks known opponent)
- `home_team_id`: user's own team (org anchor)
- `is_home_game`: determines batting order convention

Game created with `status = 'scheduled'`

2

Opponent Lineup Entry

Enter batting order for one or both teams
(home & away sides)

POST

`/bt-api/opponent-lineup/game/:gameId/bulk`

`opponentLineupService.bulkCreatePlayers()`

POST

`/bt-api/opponent-lineup/game/:gameId`

`opponentLineupService.createPlayer()`

DATABASE

opponent_lineup

INSERT

`game_id, player_name,
batting_order, position, bats
(R/L/S), team_side (home/away),
is_starter, inning_entered`

Auto-link (fire & forget): After each player insert, `_maybeAutoLinkBatter()` runs async: finds or creates a `batter_scouting_profiles` record, sets `opponent_team_id` if available, and inserts a row in `opponent_lineup_profiles` linking this game appearance to the persistent profile.

Auto-Linking Chain

- `batter_scouting_profiles`: GET or CREATE by (team_id, opponent_name, player_name)
- `opponent_lineup_profiles`: INSERT link (lineup_id → profile_id)
- `batter_scouting_profiles`: UPDATE `opponent_team_id` if null
- Substitutions: old player gets `replaced_by_id` set; new player inserted with `is_starter=false`

Pitcher Registration

- `opposing_pitchers`: INSERT pitcher per game (pitcher_name, throws, team_side)
- Auto-links to `opponent_pitcher_profiles` via `_maybeAutoLink()`
- `opponent_pitcher_profiles`: GET or CREATE by (opponent_team_id, normalized_name)

Lineup saved; scouting profiles created/linked in background

3

Game Start

User presses "Start Game" — transitions to live charting view

POST

`/bt-api/games/:id/start`

`gameService.startGame()`

DATABASE (TRANSACTION)

games

UPDATE

`status='in_progress',
current_inning=1, inning_half='top'`

innings

INSERT

`game_id, inning_number=1,
half='top', batting_team_id,
pitching_team_id,
is_opponent_batting=true`

Notes

- In scouting mode, `is_opponent_batting` is always true (both teams are opponents)
- First inning record created in same transaction
- UI navigates to Live Charting screen

status → 'in_progress' · Inning 1 Top created

4

At-Bat Creation

Select batter & pitcher from opponent lineup;
open at-bat

POST /bt-api/at-bats
`atBatService.createAtBat()`

DATABASE

at_bats **INSERT** `game_id,
inning_id,
opponent_batter_id
(FK →
opponent_lineup),
opposing_pitcher_id
(FK →
opposing_pitchers),
batting_order,
outs_before,
balls=0, strikes=0`

Scouting mode: Uses `opponent_batter_id` and `opposing_pitcher_id` (not the user's own team player IDs). Both are foreign keys into the opponent lineup tables.

Notes

- `batter_id` / `pitcher_id` (own team) are NULL in scouting mode
- `batting_order` auto-advances based on previous at-bat
- `outs_before` carries over from game state

↓ At-bat open

Pitch
Result

→

Count
Logic

→

ball

→

balls++

→

called_strike

/

swinging_strike

→

strikes++

→

foul

→

strikes++

only

if

strikes

<

2

→

in_play

→

no

count

change

(at-

bat

Pitch Logged

Chart each pitch: type, velocity, location, result

POST /bt-api/pitches

pitchService.logPitch()

DATABASE (TRANSACTION)

pitches **INSERT**

at_bat_id, game_id, opponent_batter_id,
pitch_number, pitch_type (FB/SL/CB/CH/...),
velocity, location_x, location_y, zone,
target_zone, pitch_result
(ball/called_strike/swinging_strike/foul/in_play),
balls_before, strikes_before,
team_side='opponent'

at_bats **UPDATE** balls, strikes (recalculated
from pitch_result)

batter_tendencies **UPDATE** is_stale=TRUE
(invalidates cached
tendencies for this
batter)

Scouting mode: team_side='opponent' always. Pitcher
auto-profile update (pitcher_pitch_types) is skipped
when pitcher_id is NULL (as it is in scouting).

ends
next
step)

Zone

Encoding

→

Zones

1-

9:

strike

zone

grid

(3×3)

→

Zones

11-

14:

ball

zones

(up/down/in/out)

→

location_x/y:

pixel

coords

(0-

100

scale)

6

At-Bat Completion

Record the at-bat result: out, hit, walk, HBP, etc.

POST /bt-api/at-bats/:id/end
`atBatService.endAtBat()`

DATABASE

at_bats **UPDATE**

result
(K/BB/HBP/single/double/triple/HR/...),
outs_after, **rbi**, **runs_scored**,
ab_end_time=NOW()

games **UPDATE** **home_score** /
away_score (if
runs_scored > 0),
base_runners (JSONB,
runners after play)

↓ Optional: baserunner events

Result Types

→ Strikeouts: K (swinging), Kc (called)

→ Hits: single, double, triple, HR

→ Outs: groundout, flyout, lineout, popout, FC, DP

→ Reaches: BB, IBB, HBP, error

7

Baserunner Events (optional)

Stolen bases, wild pitches, passed balls, balks, caught stealing, pickoffs

POST /bt-api/baserunner-events
 baserunnerEventService.recordEvent()

DATABASE (TRANSACTION)

baserunner_events **INSERT**

game_id, inning_id, at_bat_id, event_type
 (stolen_base/wild_pitch/passed_ball/balk/caught_stealing/pickoff),
 runner_base (first/second/third), runner_to_base, out_recorded,
 outs_before, outs_after

games **UPDATE** base_runners (JSONB) –
 move or remove runner

Advancement Events

→ stolen_base, wild_pitch, passed_ball, balk → runner advances

→ base_runners JSONB updated: { first, second, third }

Out Events

→ caught_stealing, pickoff → out_recorded=true, outs_after++

→ interference → runner awarded base

After 3 outs → advance inning

8

Inning Advancement

End of half-inning → flip sides or start new inning

POST
 /bt-api/games/:id/advance-inning
 gameService.advanceInning()

DATABASE (TRANSACTION)

games **UPDATE** inning_half
 (top→bottom or
 bottom→top),
 current_inning++
 (on bottom→top
 transition),
 base_runners={} (reset),
 outs=0 (reset)

innings **INSERT**

New half-inning record:
 inning_number, half (top/bottom),
 batting_team_id,
 is_opponent_batting

Flow

→ Top → Bottom: same inning number, teams swap

→ Bottom → Top: inning++ (e.g., end of 3rd → top of 4th)

→ base_runners JSONB reset to empty object

→ After final inning, use "End Game" instead

Repeat at-bat loop for new half-inning → after final out of final inning:

9

Game End

User ends the game — all charting complete

POST /bt-api/games/:id/end
`gameService.endGame()`

DATABASE

games **UPDATE**
`status='completed',`
`final_home_score, final_away_score`

Notes

- Simple status transition — no cascade
- Scores already maintained throughout game
- Triggers downstream scouting summary (step 10)

status → 'completed' · Post-game processing begins

10

Scouting Summary Generation

Performance summary with AI-generated narrative for all charted batters/pitchers

POST
 /bt-api/performance-summaries/generate
`performanceSummaryService.generateScoutingSummary()`

DATABASE

performance_summaries **UPSERT**
`source_type='scouting',`
`source_id=game_id,`
`pitcher_id=NULL, game_info JSONB`
`{opponent_name, scouting_home_team,`
`game_date}, stats JSONB`
`(aggregated), narrative TEXT (AI-`
`generated)`

Unique constraint: One scouting summary per game
 (idx_performance_summaries_scouting WHERE source_type='scouting').
 Generates AI narrative via Claude API summarizing pitch tendencies and at-bat patterns for all scouted players.

Summary Content

- Aggregates all pitches across all at-bats in game
- Pitch mix by type, zone heat maps, count tendencies
- Per-batter: total pitches seen, K/BB/hit outcomes
- AI narrative: chase tendency, pitch selection patterns

11

Batter Tendency Calculation

Recalculate per-batter analytics from all historical game data (triggered on demand or by stale flag)

POST

/bt-api/scouting/profiles/:profileId/calculate-tendencies
scoutingService.calculateTendencies()

DATABASE

pitches **SELECT**

All pitches via: pitches → at_bats → opponent_lineup_profiles WHERE profile_id = ?

batter_tendencies **UPSERT**

chase_rate, watch_rate,
early_count_rate,
first_pitch_take_rate,
breaking_chase_rate,
zone_tendencies JSONB,
total_pitches_seen, total_at_bats,
is_stale=FALSE,
last_calculated_at=NOW()

Calculated Metrics

- **Chase Rate**: swings outside zone / pitches outside zone
- **Watch Rate**: takes inside zone / pitches inside zone
- **Early Count Rate**: swings on 0-0, 1-0 counts
- **First Pitch Take Rate**: takes on first pitch
- **Breaking Chase Rate**: chases on SL/CB/CH outside zone
- **Zone Tendencies**: swing/take/whiff rates per zone (1-9)

Auto-Detected Insights

- High confidence (>35% chase) → "Chase outside pitch"
- High confidence (>40% breaking chase) → "Breaking ball weakness"
- High confidence (>50% early aggression) → "First-pitch hacker"

12

Opponent Team & Pitcher Profile Updates

Persistent opponent intelligence updated after game completes

GET /bt-api/opponents/:teamId **GET**

opponentTeamService.getTeam()

GET

/bt-api/opponents/:teamId/pitchers
opponentTeamService.getPitchers()

/bt-api/games/:id/opponent-roster

gameService.getOpponentRoster()

Returns known pitchers & batters from this game's opponent team for autocomplete in future games

DATABASE (UPDATED BY PRIOR AUTO-LINKS)

opponent_teams **SELECT**

name, normalized_name,
games_played, last_game_date

opponent_pitcher_profiles

SELECT pitcher_name, throws,
games_pitched,
last_seen_date

batter_scouting_profiles

SELECT player_name, bats,
opponent_team_id

Persistent Intelligence

- Opponent teams persist across seasons
- Pitcher profiles accumulate games_pitched
- Batter profiles accumulate total_at_bats
- Tendencies available for pre-game scouting reports in future games

DATABASE SCHEMA — KEY TABLES

games

```

id UUID PK
charting_mode VARCHAR ← 'scouting'
scouting_home_team TEXT
opponent_team_id UUID FK
home_team_id UUID FK → teams
opponent_name TEXT
status VARCHAR
current_inning INT
inning_half VARCHAR
base_runners JSONB
home_score INT
away_score INT
is_home_game BOOL
created_by UUID FK → users
    
```

opponent_lineup

```

id UUID PK
game_id UUID FK → games
player_name TEXT
batting_order INT
position VARCHAR
bats VARCHAR (R/L/S)
team_side VARCHAR ← home/away
is_starter BOOL
inning_entered INT
replaced_by_id UUID FK (self)
    
```

pitches

```

id UUID PK
at_bat_id UUID FK → at_bats
game_id UUID FK → games
pitcher_id UUID FK (null in scouting)
opponent_batter_id UUID FK
pitch_number INT
pitch_type VARCHAR
velocity INT
location_x FLOAT
location_y FLOAT
zone INT
pitch_result VARCHAR
balls_before INT
strikes_before INT
team_side VARCHAR ← 'opponent'
    
```

at_bats

```

id UUID PK
game_id UUID FK → games
inning_id UUID FK → innings
batter_id UUID FK (null in scouting)
pitcher_id UUID FK (null in scouting)
opponent_batter_id UUID FK
opposing_pitcher_id UUID FK
batting_order INT
balls INT
strikes INT
result VARCHAR
outs_before INT
outs_after INT
rbi INT
runs_scored INT
    
```

batter_scouting_profiles

```

id UUID PK
team_id UUID FK → teams
player_name TEXT
normalized_name TEXT
opponent_team_name TEXT
bats VARCHAR (R/L/S)
opponent_team_id UUID FK
– linked via –
opponent_lineup_profiles:
    opponent_lineup_id → profile_id
    
```

batter_tendencies

```

profile_id UUID PK FK
total_pitches_seen INT
total_at_bats INT
chase_rate FLOAT
watch_rate FLOAT
early_count_rate FLOAT
first_pitch_take_rate FLOAT
breaking_chase_rate FLOAT
zone_tendencies JSONB
is_stale BOOL
last_calculated_at TIMESTAMP
    
```

opponent_teams

```

id UUID PK
team_id UUID FK (owner)
name TEXT
normalized_name TEXT
city TEXT
state TEXT
level VARCHAR
games_played INT
last_game_date DATE
    
```

opponent_pitcher_profiles

```

id UUID PK
opponent_team_id UUID FK
pitcher_name TEXT
normalized_name TEXT
throws VARCHAR (L/R)
games_pitched INT
last_seen_date DATE
– linked via –
opposing_pitchers:
    profile_id UUID FK (nullable)
    
```

performance_summaries

```

id UUID PK
source_type VARCHAR ← 'scouting'
source_id UUID (game_id)
pitcher_id UUID FK (NULL)
game_info JSONB
stats JSONB
narrative TEXT (AI)
created_at TIMESTAMP
UNIQUE (source_type='scouting',
source_id)
    
```

RELEVANT MIGRATIONS

Migration	What It Added
015_opponent_scouting_reports.sql	batter_scouting_profiles, opponent_lineup_profiles, batter_tendencies, batter_scouting_notes
023_baserunner_event_types.sql	Extended baserunner_events with stolen_base, wild_pitch, passed_ball, balk event types
025_scouting_mode.sql	games.charting_mode='scouting', games.scouting_home_team, opponent_lineup.team_side, opposing_pitchers.team_side
026_scouting_summary.sql	performance_summaries.source_type='scouting', pitcher_id nullable, unique index per scouting game
027_opponent_teams.sql	opponent_teams registry, opponent_pitcher_profiles, opponent_pitcher_tendencies, games.opponent_team_id FK